[2] Hardness in Quadratic and Nonlinear Programming

K. G. Murty and S. N. Kabadi — *Mathematical Programming* (1987)

Some NP-complete Problems in Quadratic and Nonlinear Programming — Survey: Introduction; RQ4

The paper in one minute. Question: How difficult is it, in the worst case, to decide properties of non-convex quadratic or nonlinear optimization problems? **Approach:** Complexity-theoretic reductions connecting finite combinatorial decision problems to continuous optimization questions. **Finding:** Several deceptively simple optimization decision problems are computationally intractable in the NP-hard/NP-complete sense. **Why we care:** Non-convex optimization is difficult in general, but this result is *not* a theorem about the average neural-network training run.

1. First principles: convex versus indefinite quadratic problems

Start with the quadratic objective

$$q(x) = \tfrac{1}{2}x^\top Qx + c^\top x, \qquad x \in \mathbb{R}^d, \quad Q = Q^\top.$$

Its gradient and Hessian are

$$\nabla q(x) = Qx + c, \qquad \nabla^2 q(x) = Q.$$

If $Q \succeq 0$ and the feasible set is convex, the objective is convex: local minima are global. If Q has a negative eigenvalue, the objective has a direction of downward curvature. For example, $q(x, y) = x^2 - y^2$ has Hessian $\mathrm{diag}(2, -2)$ and a saddle at the origin. This is a *teaching example*, not the paper's reduction.

2. What does NP-complete mean in this context?

A *decision problem* asks for a yes/no answer, e.g. whether a feasible point satisfying a certain objective threshold exists. To show NP-hardness, one maps instances of a known NP-hard problem to instances of the target decision problem in polynomial time, preserving yes/no answers. NP-completeness additionally requires membership in NP for the specified encoded decision problem. This is about *worst-case instances* as input dimension grows, not a claim that every instance is difficult.

Why even local optimality is subtle. A zero gradient is necessary at an unconstrained differentiable local minimizer, but not sufficient. For $q(x, y) = x^2 - y^2$, $\nabla q(0, 0) = 0$ while $q(0, t) = -t^2 < q(0, 0)$. On constrained regions, feasibility and active constraints make local-optimality questions still more delicate.

3. The paper's method and result — careful scope

Murty and Kabadi establish computational hardness for particular quadratic and nonlinear programming decision problems. A central line of their work connects combinatorial hardness to properties of indefinite quadratic programs and local optimality; the precise reductions, input encodings, and problem statements matter. The work is cited in our survey as *background worst-case hardness*, not as a statement that SGD necessarily struggles on typical deep-learning losses.

Verification limit. The publication and an author-hosted original PDF were located, but the full reduction proof and exact theorem wording could not be extracted and checked in this environment. Consequently these notes deliberately do *not* attribute a particular reduction formula, decision direction, or theorem number to the authors. The mathematics on this page is first-principles teaching background.

4. Connection to our survey and assumptions

RQ4: This is a boundary result: general non-convex optimization can be hard, yet structural assumptions in deep-linear, low-rank, or wide-network analyses can make selected training problems tractable. Hardness of arbitrary quadratic programs does not imply that training the particular architectures in later chapters is NP-complete. **Our takeaway.** Non-convexity can hide genuinely hard decisions even in fairly simple mathematical objectives. That motivates studying which additional structures—rank, width, data assumptions, noise geometry—make neural-network training easier. **Source:** https://doi.org/10.1007/BF02592948; author copy: https://public.websites.umich.edu/~murty/np.pdf.